

Python

Les Tests Unitaires en Python

pytest • Mock • Requêtes HTTP • Bonnes pratiques

01 Introduction aux Tests

Pourquoi tester ? • Pyramide • pytest

Qu'est-ce qu'un test unitaire ?

 Un test unitaire est un programme automatique qui vérifie qu'une seule fonction produit le résultat attendu — de façon isolée et reproductible.

Analogie — Construire une voiture

```
# Avant d'assembler, on teste chaque pièce :  
  
tester_moteur()      # Le moteur tourne-t-il ?  
tester_freins()      # Les freins répondent-ils ?  
tester_direction()   # Le volant tourne-t-il ?  
  
# Si chaque pièce passe → l'assemblage a  
# plus de chances de fonctionner !
```

✓ Pourquoi c'est indispensable ?



Détecter les bugs tôt

Avant qu'ils arrivent en production



Refactoriser en confiance

Modifier le code sans tout casser



Documenter le comportement

Les tests = spécifications vivantes



Automatiser la vérification

Lance des centaines de tests en 1s

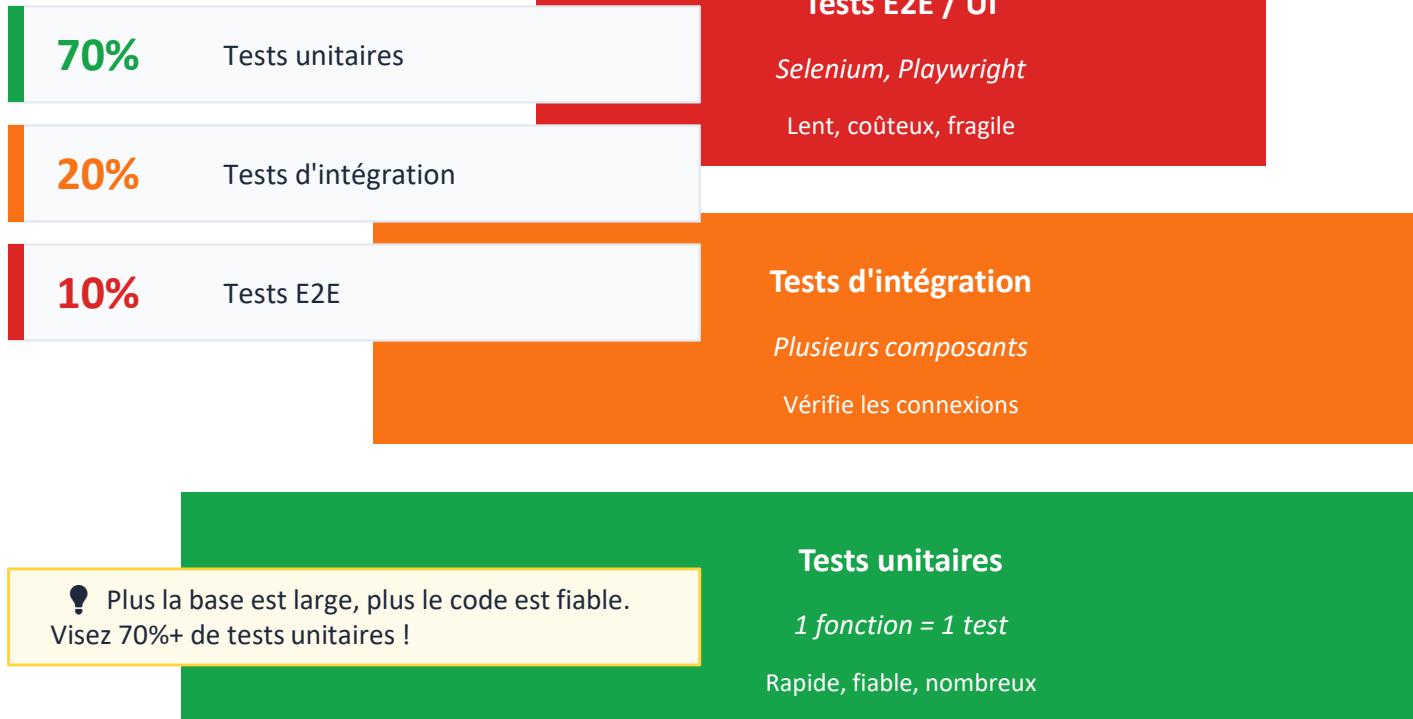


Faciliter le travail d'équipe

Chacun comprend ce que le code fait

La pyramide des tests

Règle du pouce



Pourquoi utiliser pytest ?

💡 Il existe aussi unittest (la bibliothèque standard), mais elle nécessite des classes — un concept que vous verrez dans le prochain cours. Pour l'instant, on utilise pytest qui fonctionne avec de simples fonctions !

pytest — ce qui le rend idéal pour débiter

⚡ Fonctions simples

Pas besoin de classe ni d'héritage. Une fonction `test_` suffit.

✓ assert naturel

Tu utilises le `assert` de Python. Pas de méthode spéciale à mémoriser.

🎯 Messages d'erreur clairs

pytest affiche exactement ce qui était attendu et ce qui a été obtenu.

🔧 Fixtures puissantes

`@pytest.fixture` prépare les données partagées sans classe.

📦 Paramétrage facile

`@pytest.mark.parametrize` teste plusieurs valeurs en une ligne.

🔍 Découverte automatique

pytest trouve seul tous les fichiers `test_*.py` et fonctions `test_*`.

02 pytest — Les bases

Installation • Premier test • Assertions • Cas d'erreur

pytest — Structure de base

Installation

```
pip install pytest
pytest --version    # vérifier l'installation
```

calculs.py — le code à tester

```
# Fonctions simples à tester

def additionner(a, b):
    return a + b

def diviser(a, b):
    if b == 0:
        raise ValueError('Division par zéro !')
    return a / b

def est_pair(n):
    return n % 2 == 0
```

► Lancer les tests :

```
pytest          # tous les tests
pytest -v       # mode verbeux
pytest test_calculs.py  # fichier précis
```

test_calculs.py — les tests

```
# Pas de classe ! Juste des fonctions.
import pytest
from calculs import additionner, diviser, est_pair

def test_addition_simple():
    assert additionner(2, 3) == 5

def test_addition_negatif():
    assert additionner(-1, -1) == -2

def test_nombre_pair():
    assert est_pair(4) == True
    assert est_pair(7) == False

def test_division_zero():
    with pytest.raises(ValueError):
        diviser(10, 0)
```

pytest — Assertions et vérifications

Avec pytest, tu utilises simplement le mot-clé `assert` de Python. pytest t'affiche un message d'erreur clair si l'assertion échoue.

Assertions courantes

```
# Égalité
assert additionner(2, 3) == 5
assert nom == 'Alice'

# Différence
assert resultat != 0

# Booléens
assert est_pair(4) == True
assert est_pair(7) == False
assert est_pair(4)      # même chose
assert not est_pair(7)  # même chose

# Appartenance
assert 3 in [1, 2, 3, 4]
assert 'bonjour' in message

# Type
assert isinstance(resultat, float)
```

Cas spéciaux — exceptions et flottants

```
# Vérifier qu'une exception est levée
import pytest

def test_division_zero():
    with pytest.raises(ValueError):
        diviser(10, 0)

# Vérifier le message de l'exception
def test_message_erreur():
    with pytest.raises(ValueError, match='zéro'):
        diviser(10, 0)

# Comparer des flottants
# ATTENTION : 0.1 + 0.2 != 0.3 en Python !
def test_flottant_mauvais():
    # assert 0.1 + 0.2 == 0.3 ← FAUX !
    pass

# Utiliser pytest.approx :
def test_flottant_correct():
    assert 0.1 + 0.2 == pytest.approx(0.3)
```


pytest — Fixtures (préparer les données)

Une fixture est une fonction avec `@pytest.fixture` qui prépare des données réutilisables entre plusieurs tests — sans répéter le code de préparation.

Fixture simple — données partagées

```
import pytest

# La fixture prépare les données
@pytest.fixture
def notes_eleve():
    return [15, 18, 12, 9, 14]

# Les tests reçoivent la fixture en paramètre
def test_nombre_notes(notes_eleve):
    assert len(notes_eleve) == 5

def test_meilleure_note(notes_eleve):
    assert max(notes_eleve) == 18

def test_note_moyenne(notes_eleve):
    moyenne = sum(notes_eleve) / len(notes_eleve)
    assert moyenne == pytest.approx(13.6)
```

Fixture avec nettoyage (yield)

```
# yield = sépare setup et teardown
@pytest.fixture
def fichier_temp(tmp_path):
    # -- Préparation --
    fichier = tmp_path / 'notes.txt'
    fichier.write_text('15 18 12 9 14')
    yield fichier      # fourni au test
    # -- Nettoyage (après le test) --
    fichier.unlink()   # supprime le fichier

def test_lecture_fichier(fichier_temp):
    contenu = fichier_temp.read_text()
    assert '18' in contenu

def test_fichier_existe(fichier_temp):
    assert fichier_temp.exists()
```

💡 pytest injecte automatiquement la fixture grâce au nom du paramètre. Pas besoin de l'appeler manuellement !

pytest — Paramétrer les tests

@pytest.mark.parametrize évite de copier-coller le même test pour chaque valeur. Un seul test = plusieurs cas testés automatiquement.

Sans parametrize — code répétitif ❌

```
# Mauvaise pratique : copier-coller
def test_addition_1():
    assert additionner(2, 3) == 5

def test_addition_2():
    assert additionner(-1, -1) == -2

def test_addition_3():
    assert additionner(0, 0) == 0

# → 3 fonctions pour la même logique !
```

Avec parametrize — propre et puissant ✔

```
import pytest
from calculs import additionner

@pytest.mark.parametrize('a, b, attendu', [
    (2, 3, 5),    # positifs
    (-1, -1, -2), # négatifs
    (0, 0, 0),    # zéros
    (100, -50, 50), # mix
])
def test_additionner(a, b, attendu):
    assert additionner(a, b) == attendu
    # pytest crée 4 tests distincts !
```

Résultat en console

```
pytest -v test_calculs.py

test_calculs.py::test_additionner[2-3-5] PASSED
test_calculs.py::test_additionner[-1--1--2] PASSED
test_calculs.py::test_additionner[0-0-0] PASSED
test_calculs.py::test_additionner[100--50-50] PASSED

4 passed in 0.12s
```

03 Mocks & Simulation

MagicMock • @patch • Requêtes HTTP

Qu'est-ce qu'un Mock ?

 Un Mock est un objet factice qui simule le comportement d'une vraie dépendance (API, base de données, fichier...) sans l'utiliser réellement.

Quand utiliser un Mock ?

API externe

requests.get() — pas de vraie connexion internet

Base de données

Évite de modifier les vraies données

Code lent

Simuler une tâche qui prend 30 secondes

Service payant

Éviter les appels facturés (SMS, email)

Environnement sécurisé

Fichiers système, clés secrètes

Exemple rapide — MagicMock

```
from unittest.mock import MagicMock

# Créer un faux objet
mock_db = MagicMock()

# Définir ce qu'il doit retourner
mock_db.get_user.return_value = {
    'id': 1, 'nom': 'Alice'
}

# L'utiliser comme une vraie DB
user = mock_db.get_user(1)
print(user['nom']) # 'Alice'

# Vérifier qu'il a bien été appelé
mock_db.get_user.assert_called_once_with(1)
```

@patch — Remplacer temporairement une fonction

@patch remplace temporairement une fonction dans le module testé. Elle est automatiquement restaurée après le test.

service.py — code à tester

```
import datetime

def message_du_jour():
    heure = datetime.datetime.now().hour
    if heure < 12:
        return 'Bonjour !'
    elif heure < 18:
        return 'Bon après-midi !'
    else:
        return 'Bonsoir !'
```

test_service.py — avec @patch

```
from unittest.mock import patch
from service import message_du_jour

@patch('service.datetime.datetime')
def test_message_matin(mock_dt):
    # On force l'heure à 9h
    mock_dt.now.return_value.hour = 9
    assert message_du_jour() == 'Bonjour !'

@patch('service.datetime.datetime')
def test_message_soir(mock_dt):
    # On force l'heure à 20h
    mock_dt.now.return_value.hour = 20
    assert message_du_jour() == 'Bonsoir !'
```

⚠ Règle d'or : patcher là où la fonction EST UTILISÉE, pas là où elle est définie. Ici 'service.datetime', pas 'datetime.datetime'.

Utiliser patch avec with (gestionnaire de contexte)

```
def test_message_apres_midi():
    with patch('service.datetime.datetime') as mock_dt:
        mock_dt.now.return_value.hour = 15
        assert message_du_jour() == 'Bon après-midi !'
```

Mocker des requêtes HTTP avec requests

🌐 On ne veut JAMAIS faire de vraies requêtes HTTP dans les tests. On simule la réponse avec `@patch('requests.get')`.

api_meteo.py — le code à tester

```
import requests

def obtenir_temperature(ville):
    url = f'https://api.meteo.fr/{ville}'
    reponse = requests.get(url)
    if reponse.status_code == 200:
        data = reponse.json()
        return data['temperature']
    return None

def ville_existe(ville):
    url = f'https://api.meteo.fr/{ville}'
    reponse = requests.get(url)
    return reponse.status_code == 200
```

test_api_meteo.py — mock requests

```
from unittest.mock import patch, MagicMock
from api_meteo import obtenir_temperature

@patch('api_meteo.requests.get')
def test_temperature_ok(mock_get):
    # 1. Simuler la réponse de l'API
    mock_reponse = MagicMock()
    mock_reponse.status_code = 200
    mock_reponse.json.return_value = {
        'temperature': 22.5
    }
    mock_get.return_value = mock_reponse

    # 2. Appeler la vraie fonction
    temp = obtenir_temperature('Paris')

    # 3. Vérifier le résultat
    assert temp == 22.5

    # 4. Vérifier l'URL appelée
    mock_get.assert_called_once_with(
        'https://api.meteo.fr/Paris'
    )
```

Mocker les erreurs HTTP et exceptions réseau

Tester le cas 404 — ressource introuvable

```
@patch('api_meteo.requests.get')
def test_ville_inexistante(mock_get):
    # Simuler une réponse 404
    mock_reponse = MagicMock()
    mock_reponse.status_code = 404
    mock_get.return_value = mock_reponse

    # La fonction doit retourner None
    temp = obtenir_temperature('XYZ')
    assert temp is None
```

Tester une panne réseau (exception)

```
import requests as req
import pytest

@patch('api_meteo.requests.get')
def test_erreur_reseau(mock_get):
    # Simuler une panne réseau
    mock_get.side_effect = req.ConnectionError(
        'Pas de connexion internet'
    )

    # La fonction doit lever l'exception
    with pytest.raises(req.ConnectionError):
        obtenir_temperature('Paris')
```

side_effect — simuler plusieurs appels successifs

```
@patch('api_meteo.requests.get')
def test_retry(mock_get):
    # Préparer une réponse réussie
    reponse_ok = MagicMock()
    reponse_ok.status_code = 200
    reponse_ok.json.return_value = {'temperature': 18.0}

    # 1er appel → erreur réseau, 2e appel → succès
    mock_get.side_effect = [
        req.ConnectionError('Erreur réseau'), # appel 1
        reponse_ok,                          # appel 2
    ]

    # 1er appel : exception levée
    # 2e appel : retourne la réponse ok
```

04 Exemples Complets

Gestion de budget • API météo avec pytest + mock

Exemple complet — Gestion de budget (sans classe)

budget.py — fonctions à tester

```
def deposter(solde, montant):
    if montant <= 0:
        raise ValueError('Montant invalide')
    return solde + montant

def retirer(solde, montant):
    if montant > solde:
        raise ValueError('Solde insuffisant')
    return solde - montant

def calculer_interets(solde, taux):
    return round(solde * taux / 100, 2)

def est_a_decouvert(solde):
    return solde < 0
```

test_budget.py — tests complets

```
import pytest
from budget import deposter, retirer
from budget import calculer_interets,
est_a_decouvert

def test_depot_valide():
    assert deposter(100, 50) == 150

def test_depot_invalide():
    with pytest.raises(ValueError):
        deposter(100, -10)

def test_retrait_valide():
    assert retirer(100, 40) == 60

def test_retrait_insuffisant():
    with pytest.raises(ValueError):
        retirer(50, 200)

def test_interets():
    assert calculer_interets(1000, 2.5) == 25.0

def test_decouvert():
    assert est_a_decouvert(-10) == True
    assert est_a_decouvert(100) == False

@pytest.mark.parametrize('solde, montant, attendu',
[
    (100, 50, 150), (0, 10, 10), (500, 200,
700),
])
def test_depot_parametres(solde, montant, attendu):
    assert deposter(solde, montant) == attendu
```

Exemple complet — pytest + fixtures + mock

test_meteo_complet.py — tout combiné

```
import pytest
from unittest.mock import patch, MagicMock
from api_meteo import obtenir_temperature, ville_existe

# — Fixtures —————
@pytest.fixture
def reponse_200():
    mock = MagicMock()
    mock.status_code = 200
    mock.json.return_value = {'temperature': 22.5}
    return mock

@pytest.fixture
def reponse_404():
    mock = MagicMock()
    mock.status_code = 404
    return mock

# — Tests avec fixtures —————
@patch('api_meteo.requests.get')
def test_temperature_paris(mock_get, reponse_200):
    mock_get.return_value = reponse_200
    assert obtenir_temperature('Paris') == 22.5

@patch('api_meteo.requests.get')
def test_ville_inexistante(mock_get, reponse_404):
    mock_get.return_value = reponse_404
    assert obtenir_temperature('XYZ') is None
    assert ville_existe('XYZ') == False

# — Paramétrage + mock —————
@pytest.mark.parametrize('ville', ['Paris', 'Lyon', 'Marseille'])
@patch('api_meteo.requests.get')
def test_plusieurs_villes(mock_get, ville, reponse_200):
    mock_get.return_value = reponse_200
    assert obtenir_temperature(ville) == 22.5
```

Bonnes pratiques — écrire de bons tests

1 Un test = une seule chose

Chaque test vérifie UN comportement précis. Si ça échoue, on sait exactement pourquoi.

2 Nommer clairement

`test_retrait_insuffisant_leve_valueerror()` vaut mieux que `test3()`. Le nom EST la documentation.

3 Respecter AAA

Arrange (préparer), Act (exécuter), Assert (vérifier). Structure lisible et universelle.

4 Tester les cas limites

0, valeurs négatives, chaînes vides, None — les bugs se cachent souvent aux frontières.

5 Tests indépendants

Chaque test doit pouvoir s'exécuter seul, dans n'importe quel ordre. Jamais de dépendances.

6 Mocker les dépendances

API, fichiers — toujours simulés. Un test unitaire ne touche pas au réseau ni à une vraie BD.

Résumé du cours

Test unitaire

Vérifie UNE fonction isolément. Rapide, automatique, reproductible. Base de la pyramide des tests.

Fixtures

@pytest.fixture prépare les données partagées. yield pour le nettoyage après le test.

Mock & @patch

MagicMock + @patch simule les fonctions externes. side_effect pour plusieurs appels.

pytest

Fonctions simples test_*. assert natif. Découverte auto des fichiers test_*.py. Pas de classe !

Paramétrage

@pytest.mark.parametrize pour tester plusieurs valeurs sans copier-coller le test.

Mock requests

@patch('module.requests.get') + MagicMock. Tester codes 200, 404, erreurs réseau.

Testez tôt, testez souvent — un bug trouvé en test coûte 10× moins qu'en production ! 🦋